AMERICAN INTERNATIONAL UNIVERSITY-BANGLADESH

Assignment Title:	Risk Management		
Assignment No:	04	Date of Submission:	28.05.2025
Course Title:	Software Engineering		
Course Code:	00491	Section:	**T**
Semester:	Spring 2024-25	Course Teacher:	**TONNY SHEKHA KAR**

Group Name/No.: **03**

No	Name	ID	Program
1	Monjurul Islam Nimon	23-50004-1	**CSE**
2	Prithiraj Modak	23-50007-1	**CSE**
3	Ariana Rahman Lazuk	23-50019-1	**CSE**
4	Md. Shahariyar Kabir Shanto	23-50028-1	**CSE**
5	Mst. Kamrun Nahar Omi	23-50483-1	**CSE**

Faculty use only		
FACULTY COMMENTS	**Marks Obtained**	
	Total Marks	

Risk Management

No	Risk Description	Category	Probability (%)	Impact (1–4)	RMMM
1	Delay in anomaly detection algorithm integration	Technology to be Built	40	2	**Mitigation**: Break down the algorithm into smaller, testable modules; establish clear deadlines for each sub-module. Use early prototyping and continuous integration to catch delays early. **Monitoring**: Weekly progress reports. **Management**: Reallocate resources or add temporary expert help if delays occur. Schedule buffer periods before final integration.

2	Poor quality phishing detection dataset	Technology to be Built	50	3	**Mitigation**: Combine multiple public datasets; augment data via synthetic generation techniques; perform expert validation of dataset quality. **Monitoring**: Regular quality audits on datasets; track false positive/negative rates. **Management**: Update datasets regularly; adjust models as new phishing tactics emerge; collaborate with security researchers for dataset updates.
3	SQL Injection detection false positives	Technology to be Built	40	2	**Mitigation**: Tune detection thresholds carefully; implement whitelist rules for known safe queries. **Monitoring**: Log flagged queries and conduct manual audits; analyze false positives over time. **Management**: Provide feedback loop from users/admins to improve accuracy; schedule periodic tuning sessions based on audit results.

4	Inaccuracy in DDoS detection logic	Technology to be Built	35	2	**Mitigation**: Use diverse, high-quality training datasets; perform rigorous model validation and testing against real-world traffic patterns. **Monitoring**: Continuous model performance metrics, deploy anomaly alert system. **Management**: Schedule regular model retraining cycles; consult cybersecurity experts for validation.
5	Conflicts between anomaly and attack modules	Technology to be Built	25	2	**Mitigation**: Define clear module interfaces and data contracts; implement layered logging and tracing. **Monitoring**: Monitor logs for conflicting alerts; perform integration testing. **Management**: Hold integration review meetings; adjust detection logic to reduce false conflicts.

6	Team member unavailability	Staff Size and Experience	45	3	**Mitigation**: Cross-train team members on critical components; maintain comprehensive documentation; implement pair programming. **Monitoring**: Weekly team availability check-ins; track leave schedules. **Management**: Prepare backup plans for critical roles; hire temporary contractors if needed; adjust project timeline if key skills are lost.
7	Incompatibility with desktop OS	Development Environment	25	1	**Mitigation**: Choose cross-platform development frameworks; conduct early prototyping on all target OSes. **Monitoring**: Automated multi-OS build and test pipelines. **Management**: Maintain separate OS-specific issue tracking; prioritize bugs by impact; prepare fallback builds for critical releases.

8	UI/UX design negatively affecting usability	Business Impact	45	3	**Mitigation**: Conduct early user research; develop wireframes and prototypes; apply UI/UX best practices. **Monitoring**: Collect user feedback continuously through surveys and usability tests. **Management**: Iterate on UI design based on feedback; prioritize critical usability fixes in each sprint.
9	Miscommunication between frontend and backend teams	Staff Size and Experience	35	2	**Mitigation**: Create and maintain detailed API documentation; schedule regular sync meetings; use shared collaboration tools. **Monitoring**: Track API usage and errors; maintain shared project dashboards. **Management**: Resolve interface mismatches rapidly; encourage open communication culture; conduct joint code reviews.

10	Insufficient documentation	Staff Size and Experience	50	3	**Mitigation**: Assign documentation ownership; maintain internal wiki updated in real-time; use templates. **Monitoring**: Conduct regular doc reviews and audits. **Management**: Encourage peer reviews of documentation; incentivize documentation as part of team KPIs.
11	Dependency failure (library/API updates)	Development Environment	25	1	**Mitigation**: Use version locking in dependency management; track release notes of dependencies. **Monitoring**: Run automated tests on dependency updates. **Management**: Prepare fallback code paths; maintain communication with dependency maintainers; schedule update windows.
12	Lack of testing automation	Development Environment	45	3	**Mitigation**: Develop automated unit, integration, and regression tests. **Monitoring**: Monitor test coverage and CI build success rates. **Management**: Allocate time for test script maintenance; train developers on testing frameworks.

13	Privacy compliance violation (e.g., GDPR/data handling)	Business Impact	20	1	**Mitigation:** Follow privacy-by-design principles; anonymize sensitive data; consult legal advisors. **Monitoring:** Conduct privacy audits and compliance checks. **Management:** Maintain documentation of compliance efforts.
14	Environment setup issues for new developers	Development Environment	50	3	**Mitigation:** Create automated environment setup scripts; use containerization. **Monitoring:** Track setup issues reported by new joiners. **Management:** Maintain detailed onboarding docs; assign a mentor to assist newcomers.
15	Large attack database inflates processing time	Product Size (PS)	30	3	**Mitigation:** Use scalable indexing, prune outdated data, implement pagination. **Monitoring:** Track response time and database growth with performance dashboards. **Management:** Schedule regular DB optimization, review thresholds for data archiving.

16	Inability to provide uptime guarantees during attacks	Business Impact	20	4	**Mitigation:** Use load balancers, redundant servers, and auto-scaling infrastructure. **Monitoring:** Set up real-time health checks, uptime alerts, and DDoS detection tools. **Management:** Maintain an incident response team, conduct periodic stress tests and failover drills.
17	Lack of DevSecOps experience	Staff Size & Experience	60	3	**Mitigation:** Offer training on CI/CD security tools, pair junior staff with experienced mentors. **Monitoring:** Track secure coding compliance and vulnerability scan results. **Management:** Establish DevSecOps SOPs, assign roles for secure deployment management.
18	Conflicts between versions used in dependencies	Third-party Dependencies	50	3	**Mitigation:** Use dependency management tools, lock versions via manifest files. **Monitoring:** Automate checks with SCA tools. **Management:** Maintain a version policy, schedule audits, and plan dependency updates regularly.

Impact Scale:

1- catastrophic
2- critical
3- marginal
4- negligible